BabyScholar HW2: guided test generation and evaluation

November 8, 2026 · Branch: `hw2/initial-baseline`

Pranav Anappindi · CS593 ASE

AI disclosure: The coding agent drafted this report and implemented the tests and documented fixes under the author's direction. The reflection in Section 5 was revised with AI assistance from the author's explicit feedback and recorded interventions.

1. Test Generation Approach

BabyScholar is a React and Cloudflare Worker research workspace for arXiv search, a paper library, PDF extraction, metadata, summaries and questions. The objective was to evaluate generated tests and guide improvements using actual failures, coverage and source contracts. Coverage measures execution; it cannot establish assertion quality.

The inherited initial suite had 24 deterministic Node/Vitest tests in two files: 13 identity/link cases and 11 question-request cases. These covered modern and legacy arXiv identifiers, version normalization, unsafe links, cross-paper mismatch, question shape, normalized identity and UTF-8 limits. The preserved initial instruction asked the agent to "make the initial assessments and commit and changes to the repo. I.e the baseline eval code," including profiling and coverage. The recorded assigned implementation task was to inspect BabyScholar, preserve existing work, implement the initial HW2 baseline evaluation, assess and test it, commit the changes, avoid fabricated results, and report specifics and limitations. The accompanying generation context called for a separate deterministic Node Vitest suite with explicit oracles, all-source coverage including untested files, separate legacy-check outcomes and offline operation. The agent chose identity and question-request validation for the first 24 cases; the preserved request did not prescribe that count or those particular test titles. The subsequent request and rubric required two separate guided improvements, preserved evidence and failure diagnosis.

A workflow extension targeted the previously uncovered UI and arXiv service. **dot suggested the arXiv testing seams**; the author relayed that suggestion. This corrects the prior report's attribution without rewriting archived records. The final backend request asked for deterministic provider/handler tests and investigation of two source-derived hypotheses.

The final suite has **462 tests in 13 files**: 402 Node tests and 60 React tests in jsdom. Tests use explicit expected values, real Request/Response/stream objects, injected fetch, fake timers and controlled response ordering. React cases assert visible outcomes through Testing Library. Backend handler cases execute real SQL and migrations through a small D1-shaped adapter over Node in-memory SQLite. This is not Cloudflare D1 emulation. No real credentials, paid calls, or live OpenAI/arXiv requests are used. Existing offline browser and Worker checks provide separate integration evidence.

2. Initial Test Evaluation

The Windows reproduction started from initial-test commit `e2f5c97`. `npm.cmd ci` completed, 24/24 tests passed, and 17/19 baseline stages passed; the aggregate baseline exited 1. Initial source coverage was:

Metric	Covered / total	Coverage
Statements	91 / 2960	3.07%
Branches	64 / 2595	2.46%
Functions	13 / 533	2.43%
Lines	86 / 2444	3.51%

The metadata verifier expected prompt v2 rather than current v4 and mandatory matching quotes. Its bibliographic fields, hash and page selection matched. The Windows browser waited for obsolete summary progress wording. A historical Linux browser-discovery failure was a separate environment limitation. These were not established product defects. **No existing product bug was confirmed in the initial evaluation or rounds 1 and 2.**

The initial tests correctly checked their narrow contracts, including explicit expected IDs rather than agreement between two implementations. Some rejection assertions checked only status, and the suite missed streaming, recovery, source selection, provider transport, persistence and observable UI behavior. High coverage of a small parser did not imply broad application coverage. The initial assessment links raw logs and per-file coverage; original Linux evidence remains separate from this Windows comparison.

3. Iterative Test Improvement

Round 1: input boundaries and output contracts. The recorded guidance targeted “actual bytes over dishonest Content-Length, stalled/erroring streams, the shared 10-second read deadline, cancellation that never settles, and released stream locks.” The agent added 34 request/stream/upload cases and 48 output-validation cases: 106 tests passed and line coverage reached 12.31%. Assertions included both sides of byte limits, a fixed independent PDF hash, Unicode code points, nullable metadata and ordinary summary/answer prose. The app does not require citations for new generated output; stale verifiers were corrected accordingly. The baseline passed 18/19 stages, with a later persisted-library browser timeout.

Five selected artificial regressions were tested in disposable copies. Two were detected. Lossy UTF-8 decoding, retrying a transport failure and a source budget exceeded by one byte survived. This feedback exposed weak assertions rather than three discovered product bugs.

Round 2: recovery and surviving-fault feedback. The second guidance directed recovery/source-budget tests and invalid UTF-8 “inside otherwise valid JSON strings.” It added 27 recovery, 20 source-selection and 10 encoding cases. These cover deadlines, remaining correction budgets, at most two attempts, ownership checks, no retry of transport failures, intact Unicode and unchanged persisted extraction. A test syntax error initially stopped the encoding file from loading; that failed run is retained. After correction, 163 tests passed, line coverage was 16.12%, and all five selected faults were detected.

The weak round-1 fixture was literally:

```
{ name: 'invalid UTF-8', bytes: new Uint8Array([0xff]) }
```

Its assertion expected `invalid_request`/400. A lossy decoder turns that bare byte into a replacement character, which is still invalid JSON; therefore the test passes even when UTF-8 validation is broken. The round-2 helper instead inserts invalid bytes between a valid JSON prefix and suffix:

```
const [prefix, suffix] = json.split('MARKER');
controller.enqueue(encoder.encode(prefix));
controller.enqueue(insertion);
controller.enqueue(encoder.encode(suffix));
```

Here `json` contains a string such as `{"question":"A_MARKER_B"}`. Replacement decoding would produce valid JSON, so rejecting it distinguishes strict UTF-8 validation from ordinary JSON syntax checks. A correctly encoded U+FFFD is a positive control. These excerpts come from the preserved round-1 stream test and the round-2 encoding test.

Round 3: observable UI workflows and arXiv resilience. The extension verified `d66051c`, installed dependencies and reproduced 163 passing tests with 18/19 baseline stages. App and DocumentWorkspace accounted for 807/2444 measured lines (33.02%) at zero Vitest coverage; arXiv added 214 uncovered lines. Separate legacy browser checks had already exercised these features.

The pre-implementation instruction included: "uncertain-save GET reconciliation without automatic POST retries" and "stale library/detail responses." TSX discovery and pinned DOM tooling were added. Before new tests, a frozen 163-test bridge reproduced exactly the prior coverage counts under the new configuration. The initial expansion added 37 arXiv cases and 56 component cases, yielding 256 passing tests and 55.64% lines. Actual mistakes included an invalid Atom ID fixture, a wrong button label and an incorrect assumption about automatic summary reload. They were corrected against source contracts, not by changing the product to satisfy the tests.

Seven of eight selected workflow faults were caught. The survivor rejected an exactly 10 MiB download: upload-limit tests had not covered the download reader. A download boundary pair, stale-download case and negative save-confirmation counterpart raised the suite to 260 and caught all eight faults. Line coverage stayed 55.64%; stronger oracles improved detection without adding covered lines. One real App feedback bug was independently reproduced and fixed (Section 4).

Round 4: provider and generation boundaries. The final pass verified branch/remote `69c2eff` and a clean tracked tree, preserved untracked files, ran `npm ci`, and reproduced exactly 260 tests and 1360/2444 lines. The four provider modules and three generation handlers each had zero measured coverage. The recorded guidance prioritized "public functions with injected fetch, real Response/ReadableStream objects, and fake timers" and required reproduction before any product fix.

The agent added 24 probe, 98 provider and 80 handler cases. Assertions cover successful request/output shape, safe authentication/rate-limit/model errors, malformed/incomplete/refusal responses, body deadlines, exact response-byte limits, streamed Unicode, cancellation, sanitized errors and no unintended retry. Handler tests assert persisted output and provenance, cache reuse without a key, read-only GET, concurrent claim exclusion, ownership loss, deletion during generation, failed persistence and cleanup. Metadata/summary may correct invalid output once; the question handler does not automatically retry it. Tests respect those actual differences.

The first handler run passed 69/71. One timeout test advanced fake time before asynchronous question fingerprinting had started the provider deadline; another supplied forbidden outer whitespace. Synchronizing on provider dispatch and using internal whitespace fixed the tests. Nine additional ownership, cleanup and corrupt-record cases brought the handler file to 80/80.

A typecheck also exposed four misleading green provider UTF-8 tests: tuple rows in `it.each` were spread into arguments, so a numeric first argument created NUL-filled buffers rather than the intended bytes. Named object rows and a positive JSON-structure control fixed the oracle. A single isolated lossy-decoder mutation survived the original 98-case file but failed four corrected assertions; both clean controls passed.

Coverage remained identical. The development record preserves these errors, corrections and exact snapshots. The two probe hypotheses became confirmed defects through focused before/after tests, described below.

Bounded final quality review. A final instruction asked for a representative 10–15-case review rather than another coverage round. The agent examined 15 concrete cases across component UI, providers, generation handlers and boundaries. Two weaknesses were corrected: a truncated-source fixture listed pages absent from its actual source, and a rollback test injected failure before any row was written. The first now checks consistent page metadata and actual transmitted text; the second injects failure after INSERT and verifies the row is rolled back. These strengthen four existing parameterized cases without increasing the test count. The affected 178 tests passed both before and after. No additional product bug or genuinely redundant test was found in this sample. Exact-limit acceptance and one-byte-over rejection are necessary pairs. This is not an audit of every test. Product source, dependencies and coverage configuration were unchanged.

Browser investigation and limits. Earlier round-3 diagnostics showed Vite API/static requests stalling while direct Worker requests succeeded. Draining child output did not fix it; an isolated Vite/module/proxy check passed all 21 requests. Further runs identified stale Q&A progress text, an unconfigured metadata-failure fixture and routing by removed schema names. Those harness assumptions were repaired without extending deadlines or changing the product proxy. Complete built-app preview and normal baseline browser runs passed in round 3. Earlier development-server stalls remain unexplained; subsequent passes do not prove the intermittency resolved.

4. Final Evaluation

The final suite passes **462/462 tests**. The final full baseline result is **19/19 stages passed (aggregate exit 0)**. Both focused test typechecks pass. The baseline separately typechecks/builds production browser and Worker code. The final review rerun, started November 8 at 7:59 PM EST, confirms these results after the two bounded test corrections. Its comparison verifies identical per-file coverage counts, product source tree, dependency lock and Vitest configuration against round 4. Stage commands differ only in timestamped profile output paths. No failures remain in this run; the earlier intermittent browser limitation still applies.

Measure	Initial	Round 1	Round 2	Round 3	Round 4 / final review
Passing tests	24	106	163	260	462
Statements	3.07%	12.29%	15.87%	53.09%	80.32%
Branches	2.46%	10.13%	12.21%	51.44%	78.21%
Functions	2.43%	12.57%	15.38%	57.11%	79.96%
Lines	3.51%	12.31%	16.12%	55.64%	83.96%
Cumulative confirmed bugs	0	0	0	1, fixed	3, fixed

Final counts are 2380/2963 statements, 2032/2598 branches, 427/534 functions and 2053/2445 lines. The all-file CSV and machine-readable comparison preserve every phase, including the unchanged preflight and pre/post-oracle-correction runs.

Backend target	Round-3 covered lines	Final covered lines
metadata-openai.ts	0/98	98/98
summary-openai.ts	0/98	98/98
question-openai.ts	0/110	109/110
openai.ts	0/61	62/62
document-metadata.ts	0/103	94/103
document-summary.ts	0/109	101/109
document-questions.ts	0/104	97/104

This is one combined Vitest 5.1.0/V8 measurement across Node and jsdom, not averaged percentages. Scope is all `src/**/*.ts(x)` except declarations, type-only `*-types.ts` and two local probe-route modules. Scripts, CSS and fixtures are outside the denominator; actual browser/workerd execution is excluded. Round 3's configuration bridge supports the TSX change. Round 4 keeps that configuration and the dependency lock byte-identical. Product changes affect denominators: the App fix added one statement/function; the probe fixes add one line, two statements and three branches. The scope remains consistent, but the source and denominators are not literally identical across all rounds. Counts and deltas are disclosed rather than hidden.

Confirmed bug 1: contradictory save feedback. After an unreliable save response, GET returned the authoritative saved paper. App showed a disabled Saved button while also reporting that the save could not be confirmed. Four response variants reproduced it on unchanged source. The assertion that failed was exactly:

```
expect(screen.queryByRole('alert')).not.toBeInTheDocument();
```

The log reported the unexpected alert: “The save could not be confirmed after reloading the library. You can try again.” Success-only assertions would have missed that contradiction. The old lookup was:

```
const confirmed = savedPaperFor(papers, identity);
```

It normalized an already-prefixed identity twice. The separate fix is:

```
const confirmed = papers.find((paper) => arxivSourceIdentity(paper.arxivId) === identity);
```

The regression is in app-workflows.test.tsx, titled `reconciles %s with GET and shows a confirmed save without a contradictory error` for `lost response`, `malformed success`, `wrong paper` and `non-application error`. The bug evidence records the oracle and failing run `2026-11-08T22-25-47.425Z-app-first`: all four variants failed the absent-alert assertion above. A fifth failure was a wrong button selector in another test, not a fifth product failure. A negative counterpart rejects a different paper as confirmation, and regression cases require only one POST. This is a feedback/state bug; no data loss or duplicate persistence is claimed. Fix: `852eb2c`; workflow tests: `2c83268`.

Confirmed bug 2: local probe body deadline. The regression in openai-probe.test.ts is titled `keeps the deadline active while consuming a stalled response body`. The probe cleared its timer after receiving headers, before reading JSON. With a real abort-aware stalled response stream, advancing a 20 ms fake budget

failed `expect(signal.aborted).toBe(true)` with "expected false to be true"; the same stalled-before-headers case already passed. Retaining the timer through body consumption fixes this. It assumes an AbortSignal-aware fetch; arbitrary non-cooperating injected promises are not claimed to be forcibly interrupted.

Confirmed bug 3: local probe null-response classification. In the same file, `propagates null JSON to the caller as malformed response, not network failure` failed with actual `network_failure` rather than expected `malformed_response`. The companion `rejects unusable successful payload 0 as a classified error` received a TypeError instead of IntegrationError. HTTP 200 JSON `null` had reached a status access without a guard. A guard now returns `malformed_response`. The route already caught the exception; this was not a Worker-process crash. The same 24 tests passed 21 before and 24 after the fix, with three failing assertions identifying the two defects. Before/after evidence is preserved. The generation adapters already had body deadlines/null guards; these findings concern the local connectivity probe. Both probe fixes are in `5285152`; backend tests: `89afbd3`. The preserved pre-fix run is `2026-11-08T23-38-14.457Z-r4-probe-before`; the same assertions pass in `2026-11-08T23-38-38.111Z-r4-probe-after`.

Artificial faults and remaining limits. The original five selected faults progressed from 2/5 to 5/5 detections. The eight workflow faults progressed from 7/8 to 8/8. Round-4 reruns: **5/5 original and 8/8 workflow faults detected, each with a 462-test passing control**. The additional selected provider UTF-8 fault progressed from surviving to four failing assertions. These deliberate examples are not discovered product bugs, an exhaustive mutation score, or an estimate of detection against arbitrary defects.

About 16% of source lines remain uncovered in the measured suite, including much routing, storage and PDF parsing. Even fully covered lines may have weak or missing assertions. SQLite tests do not prove distributed D1 behavior, R2, hosted authentication or deployment. jsdom does not prove layout or real browser downloads; the preview fixture forced download permission. Metadata/summary replacement UTF-8 decoding differs from question's strict policy; this pass does not impose or claim a new shared policy. No live model quality, production load, OCR or performance improvement was established. CPU profiles/timings are retained, but changing workloads and local load preclude a speedup claim.

Dependency audit findings (separate from test-discovered bugs). The preserved November 8 read-only `npm audit` reports eight affected package entries: four moderate and four high. The moderate entries are `drizzle-kit` 0.31.11, `@esbuild-kit/esm-loader` 2.6.5, `@esbuild-kit/core-utils` 3.3.2 and nested `esbuild` 0.18.20. The high entries are `wrangler` 4.148.0, `miniflare` 5.20261006.0-alpha, `sharp` 0.35.4 and `undici` 7.29.1. The count includes dependency-chain propagation, so it is not eight independent application defects. The esbuild finding concerns its development server; sharp and undici have image-processing dependency and HTTP/TLS-related advisories respectively.

All eight entries are classified as development dependencies; a separate `npm audit --omit=dev` reports zero vulnerabilities. This does not establish that the deployed application or development environment is safe, and no exploitability test was performed. These are known dependency-audit findings, not additional bugs discovered by the HW2 test suite. The committed dependency audit records the package details. The tested lockfile was preserved rather than applying a forced upgrade immediately before submission; dependency remediation requires separate compatibility testing.

5. Reflection

I had to repeatedly push for more work rather than accept the first passing result as sufficient. After the first two improvement rounds, dot was ready to call the test work done even though line coverage was only 16.12% and no existing product bug had been confirmed. I challenged that conclusion and asked how we could use the remaining five hours to do a good job rather than make minimal progress. That led to the UI and arXiv round, which reached 55.64% coverage and found the save-confirmation feedback bug.

I had to push again after that round. With roughly three hours left, I explicitly reminded dot that it had been willing to stop before the previous revision and asked what else was worth doing. The resulting backend pass reached 83.96% line coverage and reproduced two more defects in the local connectivity probe. This repeated motivation on my part mattered: the assistant was too eager to call it a day, and substantial gaps remained each time it initially treated the work as ready. The extra rounds were not just cosmetic increases in test count; they exercised important missing behavior and found real defects.

My role was to challenge the stopping point and demand stronger evidence; dot helped identify targets and draft guidance, and Codex implemented and ran the tests. The test implementations described here were produced by Codex. I also supplied a terminal transcript of a fresh local verification: dependency installation, 462 passing tests, 83.96% line coverage, both focused test typechecks and all 19 baseline stages passing. This is a user-supplied execution record, not a claim that dot independently reran the suite. Its baseline output is `artifacts/baseline/2026-11-09T00-30-17.544Z`.

The agent handled explicit contracts well: byte boundaries, ownership, read-only reconciliation, failure classification and observable UI outcomes became useful tests. Extra context mattered. The citation/version warning prevented obsolete requirements from becoming oracles; dot's seam suggestion made deterministic arXiv failures and time control practical; the two probe hypotheses focused investigation on reproducible defects.

The agent also produced incorrect assumptions and superficially green tests. Bare invalid UTF-8 bytes, and later the tuple-parameterization mistake, allowed rejection for the wrong reason. A syntax error, wrong accessible name, invalid Atom fixture and premature fake-clock advancement required correction. Source review, typechecks and controlled faults supplied evidence that a passing count and higher coverage could not. The repeated UTF-8 mistake shows that prior feedback does not guarantee correct application in a new context.

Specific guidance was more useful than asking only for more tests: put bad bytes inside valid JSON, assert absence of contradictory feedback, test download limits separately from upload, and verify ownership before persistence. The narrow product fixes were separated from test changes. Failed runs and snapshots remain reviewable. Human guidance set scope, corrected attribution and established the deadline; implementation, fixture repairs, execution and the initial analysis were agent-produced. The reflection above incorporates my explicit criticism of the assistant's stopping decisions. Passing tests and a polished report were not enough grounds to stop while important untested workflows and time for a focused investigation remained.

6. GitHub Repository

Repository: panappin/babyScholar. Branch: hw2/initial-baseline. Final suite: tests/baseline (13 test files, three helpers). Instructions, results, per-file coverage, snapshots and SHA-256 manifests remain in docs/hw2/rounds. The round-4 assessment indexes the backend evidence; the final quality review records the bounded sample and corrections. The author's independent baseline rerun is preserved in its own archive. Its existing artifacts confirm 19 passing stages and the reported coverage at `57d2e1e`; the separate installation and typecheck transcript is described by the author, not claimed as an agent-executed run.

Tested environment: Node 24.11.1, npm 11.6.2, Windows x64, Intel i9-13900HX. Use the locked dependencies. Node supplies the in-memory SQLite implementation; its experimental warning is retained in logs and is not a test failure. From this branch:

```
npm.cmd ci
npm.cmd test
npm.cmd run test:coverage
npm.cmd run test:workflows:types
npm.cmd run test:providers:types
npm.cmd run baseline
```

On other platforms use `npm`; the legacy browser script discovers Chrome/Edge in supported Windows paths. Coverage writes HTML, JSON and LCOV to `coverage/baseline/`. The baseline creates a timestamped `artifacts/baseline/` directory with stage logs, exits, environment and profiling. It continues through failed stages and exits nonzero if any fail. Historical evidence must not be overwritten. The two focused typechecks cover the newly added workflow/provider files and helpers; the baseline checks production projects.

Additional reproduction commands:

```
node scripts/baseline/compare-hw2.mjs
node scripts/baseline/compare-workflows.mjs
node scripts/baseline/compare-backends.mjs
node scripts/baseline/fault-probes.mjs round-4
node scripts/baseline/workflow-fault-probes.mjs refined
node scripts/baseline/provider-utf8-probe.mjs
npm.cmd run build:frontend
node scripts/baseline/browser-experiment.mjs preview
node scripts/baseline/render-report.mjs
```

Faults run in isolated source/test copies with passing clean controls. Preview uses the built app, disposable local Worker/D1/R2 state and loopback provider fixtures; it is separate from the normal development-server baseline. No credentials or paid requests are required. The PDF renderer converts this editable report to `docs/hw2/report.pdf` using locked Marked and local Chromium, validates local link targets and emits portable GitHub URLs. The PDF verification record records rendering and page checks.

The code, evidence and revised report are published only on the HW2 branch. The author retains responsibility for the final review and assignment submission.